

Technical Interview Exercise

The engineering team would like you to complete the following technical assignment.

This exercise is intended to ensure that candidates are able not only to finish the task at hand but also to provide the information we need to guide a future technical interview.

Some details of the task have been left intentionally vague to give you the opportunity to explore different options.

Fundamentals

- An order book contains both buy and sell orders (for a single instrument or market) from different accounts.
- An instrument is defined by a pair of assets, such as BTC/BRL, where one asset is quoted (or priced) in relation to the other.
- Accounts represent end users and hold the balances of the users' different assets.
- Orders have a limit price and a quantity of the asset being traded.
- If for a given order (buy or sell) there is a counterpart order in the book with an equal or better price, a match occurs and the matched quantity is removed from both orders.
- Orders are submitted from different accounts, and once a match occurs, the balances are exchanged according to the matched quantity.

Example: if a buy order for 1 BTC at 500k BRL from account A matches a sell order for 1 BTC at 500k BRL from account B, then 1 BTC will be transferred from B to A and 500k BRL will be transferred from A to B.

Additional information:

- [O que é uma ordem limitada?](#)
- [Guia completo para iniciantes](#)

Task

Using Golang, you will create a simplified Central Limit Order Book (CLOB) and a simplified matching engine to execute limit orders on it. You must also manage the balances of the different accounts that submit orders to the book.

To interact with the book and the accounts you will have to implement 2 basic operations and the matching logic:

- Place an order into a book (required)
- Cancel an order (required)

In order to implement these two operations, you will likely need to implement some type of balance control, and extra bonuses will be considered if the following supporting operations are included (all optional):

- Credit an asset to an account
- Debit an asset from an account
- Get an account's current balance
- Get the current order book for an instrument

Scoring

There is no absolute right or wrong solution; what we want to understand is how you approach the problem and which options you choose when implementing the solution.

However, we will evaluate code quality, adherence to good programming practices, and fluency in Golang. Whenever possible, please rely on the standard library before introducing external dependencies, to facilitate analysis and comparison with other solutions.

Deliverables

We expect either a public GitHub repository or a ZIP file containing the repository, plus a README.md in the root folder with instructions on how to build and run the project and how to verify that the operations work.

Any additional documentation clarifying implementation details, assumptions made, and design decisions is also welcome.